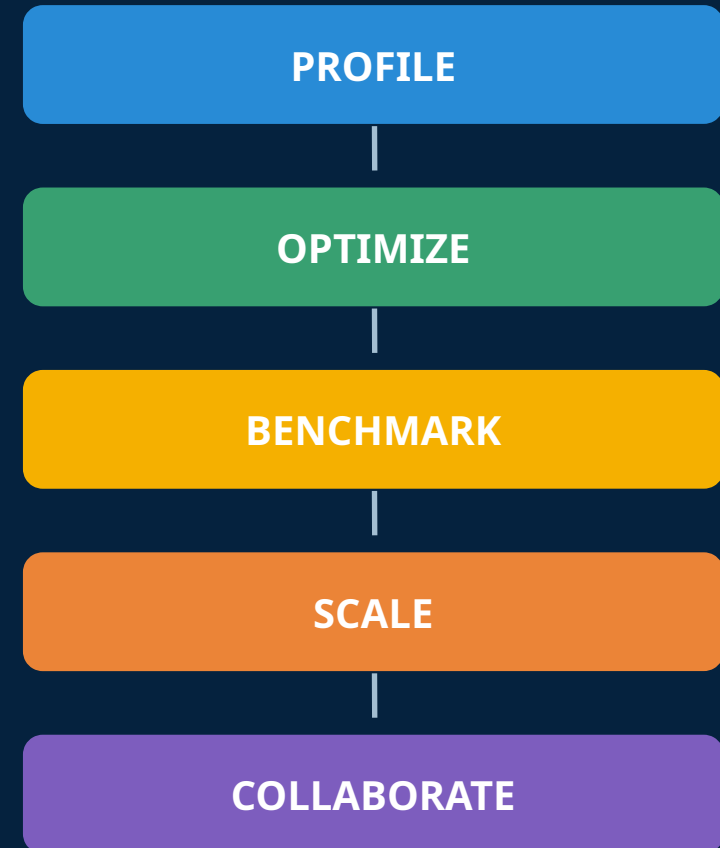


LECTURE 7

HPC PERFORMANCE OPTIMIZATION, SCALING & COLLABORATIVE SCIENTIFIC COMPUTING

From working simulations to efficient, scalable and impact-oriented research

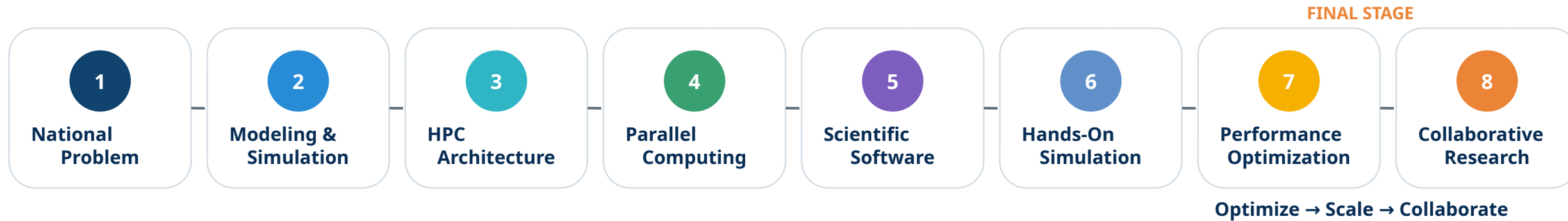
Speaker: Prof. Dr. Tassadaq Hussain



PROFILE → FIX → TEST → SCALE → IMPACT

Our Journey Through the Summer School

The final stage connects optimization with collaborative research.



"IT RUNS"

"IT RUNS EFFICIENTLY"

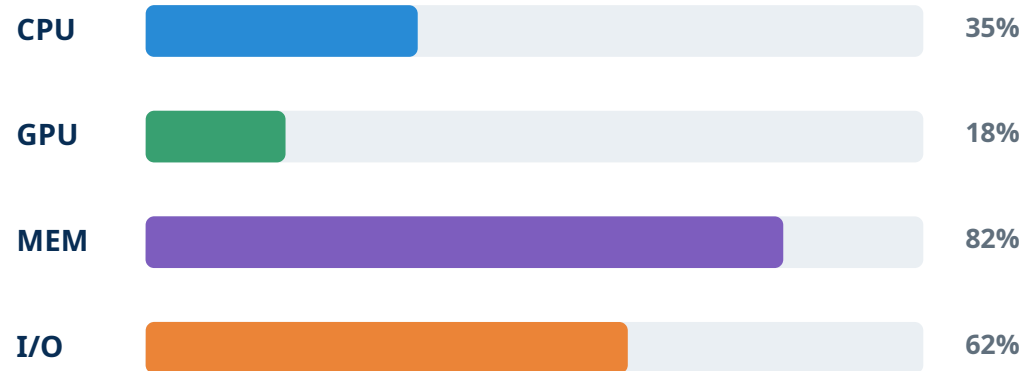
"IT CREATES IMPACT"

Performance engineering is the bridge between a working simulation and useful large-scale science.

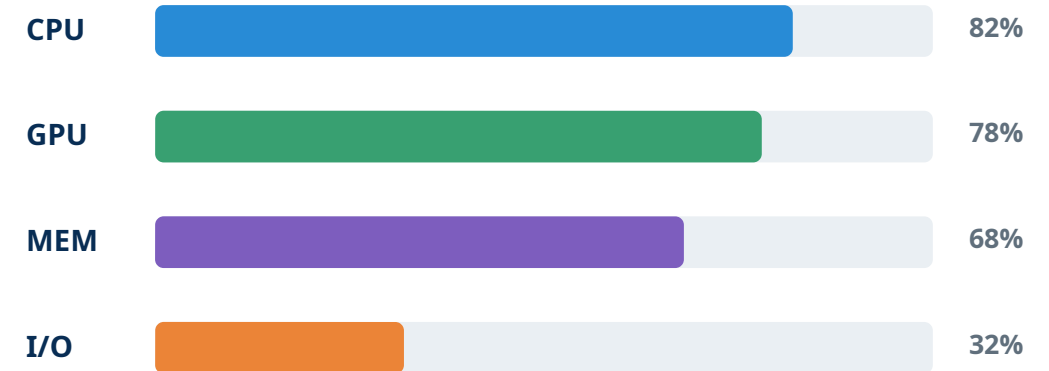
Running $\neq$ Running Efficiently

A correct simulation can still waste substantial computing resources.

WASTEFUL RUN



EFFICIENT RUN



PROFILE

FIX BOTTLENECK



Idle cores

Resources allocated but not busy



Oversized requests

Too much memory / wall time / nodes



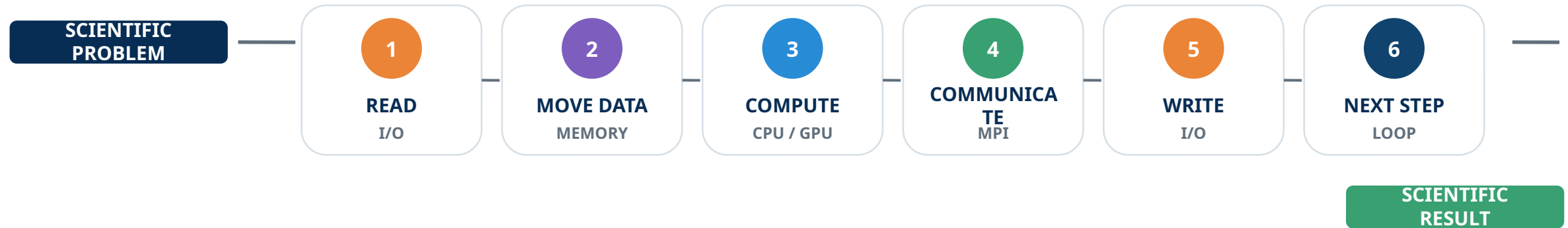
Excess communication

Scaling adds waiting instead of speed

Goal: deliver the required scientific result with the right resources, accuracy and time-to-solution.

Anatomy of Simulation Performance

A time step moves through data, compute, communication and output.



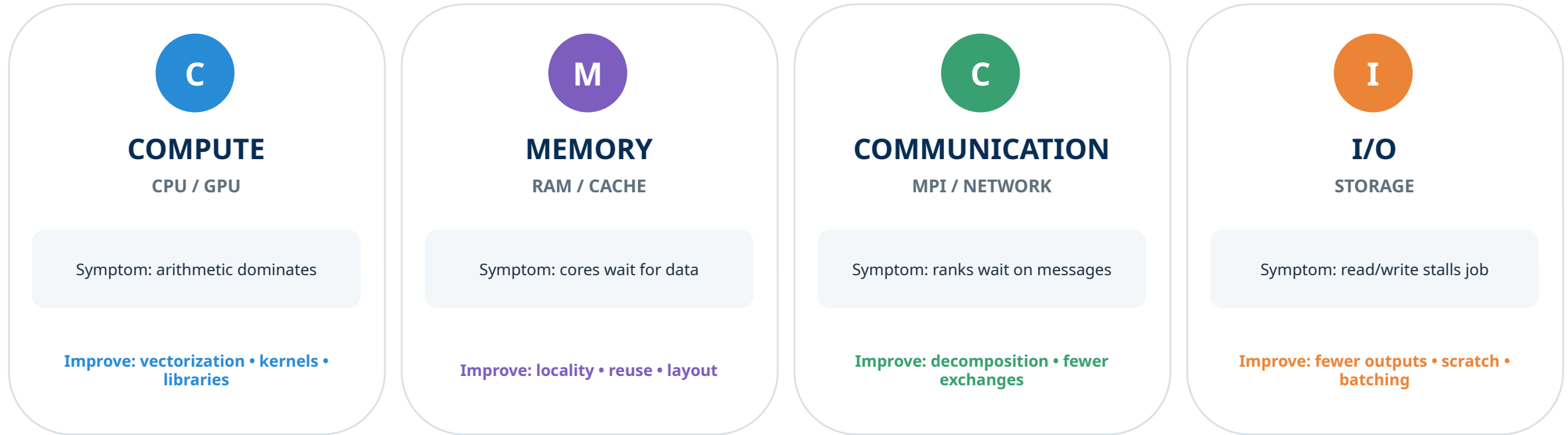
POSSIBLE BOTTLENECKS



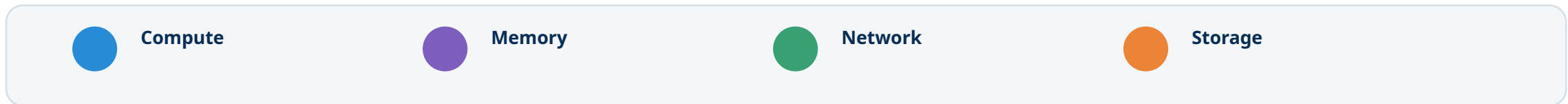
Profile at the scale you actually intend to run.

What Determines HPC Performance?

Four subsystems compete to limit time-to-solution.



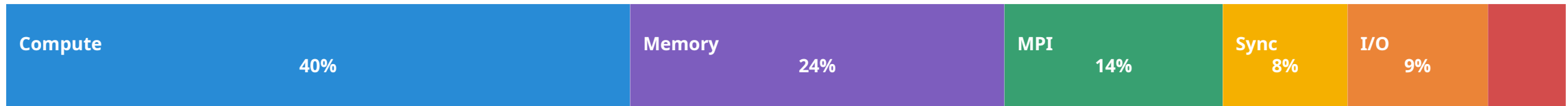
PERFORMANCE IS A BALANCE



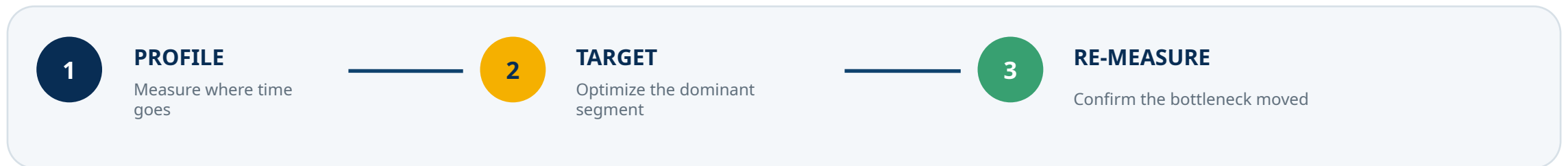
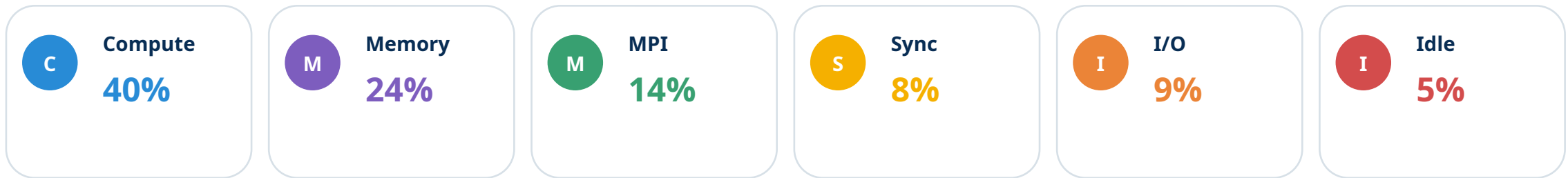
The slowest subsystem often defines the overall job.

Where Does Execution Time Go?

The first performance question is: “What is the application waiting for?”



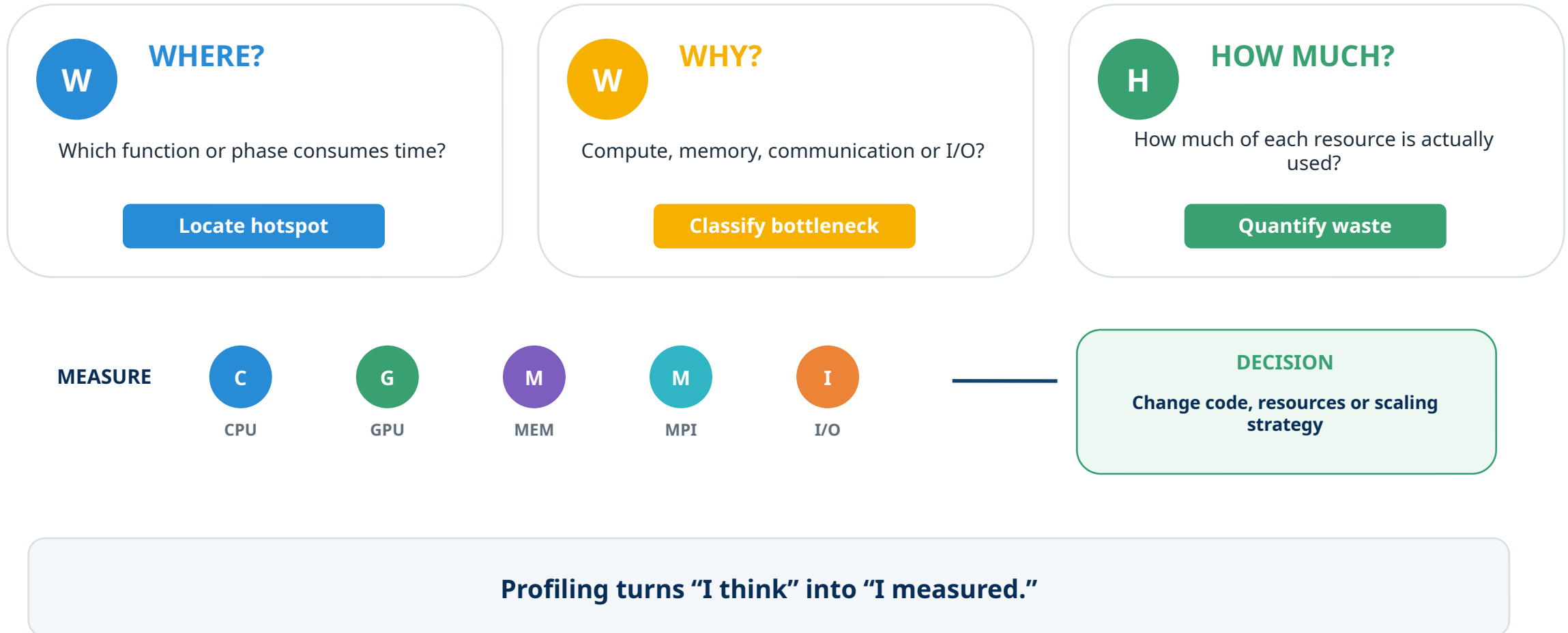
Illustrative execution-time breakdown



Optimize the bottleneck—not the most visible code.

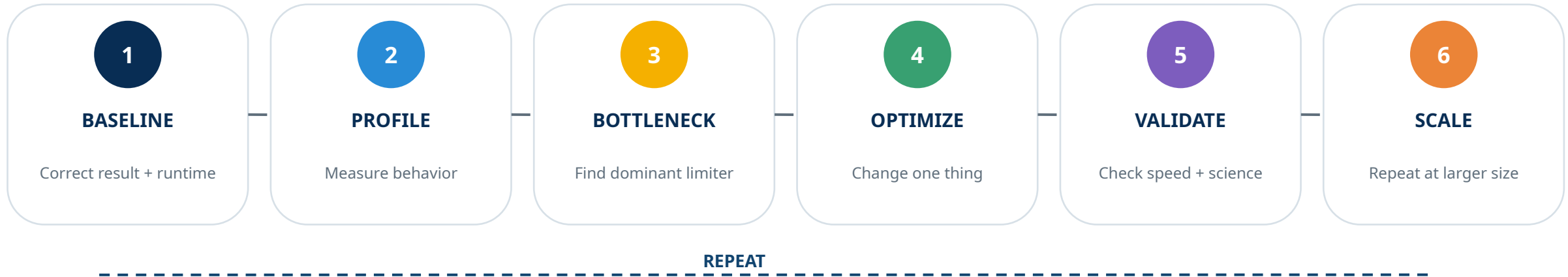
What Is Performance Profiling?

Profiling measures how the application actually uses the machine.



Golden Rule: Measure Before You Optimize

Do not optimize by intuition alone—use an evidence-driven loop.



 **DON'T** Change many settings at once or optimize without a baseline.

 **DO** Change one factor, measure again, and preserve scientific correctness.

Optimization is an experiment: hypothesis → change → measurement.

Important Performance Metrics

Use metrics that directly answer a performance question.

E
TIME

Execution time

How long
does one job
take?

T

**WORK /
TIME**

Throughput

How much
science per
hour?

C

CPU utilization

% BUSY

Are CPU cores
doing useful
work?

G

GPU utilization

% BUSY

Is acceleration
actually used?

M

Memory / bandwidth

GB / s

Is data
movement
limiting?

S

Speedup / efficiency

S, E

Does
parallelism pay
off?

A metric is useful only when it changes a decision.

Basic Profiling Tools

Start simple. Move to deeper tools only when the question requires it.

1

time / sacct

Runtime + job efficiency

Start here

2

top / htop

CPU + memory activity

Quick live check

3

nvidia-smi

GPU utilization + memory

GPU jobs

4

perf

CPU hotspots + counters

CPU bottlenecks

5

Nsight / MPI tracing

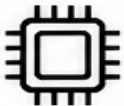
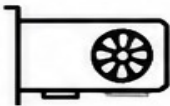
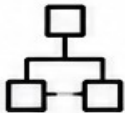
Kernel + communication timelines

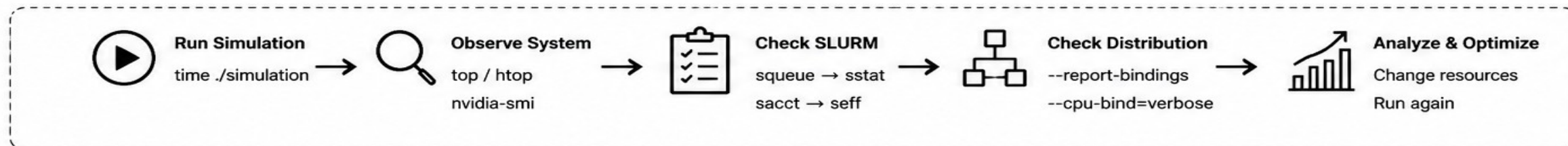
Deep diagnosis

Question → simplest tool that can answer it → evidence → optimization decision

HPC Time & Performance Measurement

From basic runtime to distributed (SLURM) information

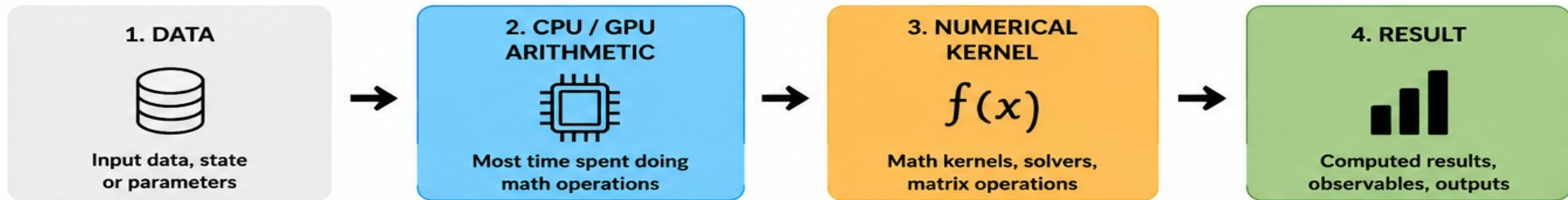
	1. BASIC RUNTIME 	2. CPU USAGE 	3. CPU PERFORMANCE 	4. GPU USAGE 	5. SLURM JOB INFO 	6. DISTRIBUTED (MPI) 
Command	<code>time ./simulation</code>	<code>top -u \$USER</code> or <code>htop</code>	<code>perf stat ./simulation</code>	<code>nvidia-smi</code> or <code>nvidia-smi dmon</code>	<code>queue -u \$USER</code> <code>sstat -j <JOBID></code> <code>sacct -j <JOBID></code> <code>seff <JOBID></code>	<code>mpirun --report-bindings</code> <code>-np N ./simulation</code> or <code>srund --cpu-bind=verbose</code> <code>./simulation</code>
What you get	• Real (wall) time • User CPU time • System CPU time	• Per-core CPU usage • Running processes • Load average	• CPU cycles • Instructions • Cache misses • Branch misses • Elapsed time	• GPU utilization • GPU memory • Power / Temp • Running processes	• Job status & runtime • Resource usage • CPU / Memory used • Efficiency (seff)	• Rank placement • CPU/core binding • Process distribution



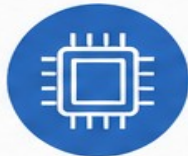
Goal: Measure → Understand → Optimize → Repeat
Measure first. Optimize based on data, not guesses.

Compute-Bound Workloads

The processor spends most of its time performing arithmetic (computations).



What is Compute-Bound?



The application performs a large number of arithmetic operations and the CPU/GPU is busy most of the time.

Performance depends on:

- Algorithm efficiency
- Vectorization & parallelism
- CPU/GPU compute capability
- Optimized libraries and kernels

How it Looks (Resource View)



CPU / GPU Utilization



Memory Bandwidth



Communication (MPI)



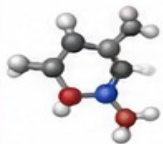
I/O Activity



Compute is the limiting factor (bottleneck).

Examples of Compute-Bound Simulations (Domains)

Molecular Dynamics (Biophysics / Chemistry)



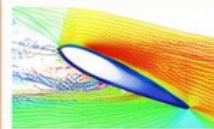
Force calculations between atoms/molecules dominate runtime.

Quantum Chemistry (Materials / Chemistry)



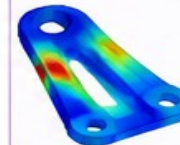
Electronic structure and integral calculations are computationally intensive.

Computational Fluid Dynamics (CFD) (Engineering / Aerospace)



Solving Navier–Stokes equations requires heavy arithmetic.

Finite Element Analysis (Structural / Solid Mechanics)



Large systems of equations and matrix operations dominate.

Astrophysical N-Body Simulations (Astrophysics / Cosmology)



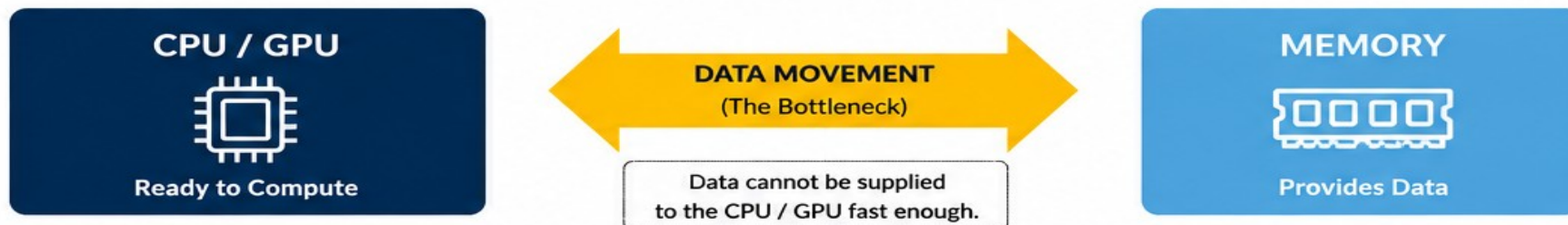
Gravitational force computations between many bodies.



Key Takeaway: In compute-bound workloads, keep the compute units (CPU/GPU) busy with better algorithms, parallelism, vectorization and optimized libraries. The goal is to do more calculations per unit time.

Memory-Bound Workloads

The processor is ready, but data cannot arrive fast enough.



What is Memory-Bound?



- The application spends most of its time waiting for data to come from memory.
- Arithmetic units are underutilized.

Causes:

- Low data reuse (poor locality)
- Large data structures
- Irregular memory access patterns
- Insufficient memory bandwidth

How it Looks (Resource View)



CPU / GPU Utilization



Low – Moderate



Memory Bandwidth



High (Near Max)



Communication (MPI)



Low – Moderate



I/O Activity



Low – Moderate



Memory bandwidth is the limiting factor (bottleneck).

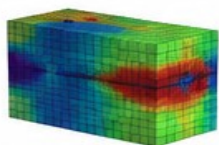
Examples of Memory-Bound Simulations (Domains)

Large-Scale Climate Models
(Atmospheric / Ocean)



Work with large grids and frequent data access.

Reservoir Simulation
(Petroleum Engineering)



Large datasets with irregular memory access.

Particle Transport / Monte Carlo
(Radiation / Neutron Transport)



Irregular memory access and low data reuse.

Genomics / Bioinformatics
(Sequence Alignment)



Large data structures and memory-intensive algorithms.

How to Execute Memory-Bound Workloads Effectively



Improve Data Locality

Access nearby data together, increase cache hits.



Use Efficient Data Layouts

Prefer contiguous memory layouts (e.g., SoA vs AoS).



Increase Cache Reuse

Block/tiling techniques, reuse data in cache.



Maximize Memory Bandwidth

Use higher bandwidth memory, optimize prefetching.



Reduce Unnecessary Movement

Avoid copying data between memory levels.



Profile and Tune

Use profiling tools to find hotspots and optimize.



Key Takeaway: Minimize data movement and maximize data reuse. Bring data closer to the compute units and feed them efficiently.

Communication-Bound Workloads

Distributed execution can become limited by data exchange between processes or nodes.



Communication (network) becomes the limiting factor when:

- Large amount of data is exchanged
- Many small messages are sent
- Frequent synchronization is required
- Network latency / bandwidth is limited

What is Communication-Bound?



The application spends a large portion of time exchanging data across nodes or waiting for other processes.

Causes:

- High volume of MPI messages
- Small message size, high message rate
- Global synchronization (barriers, gathers)
- Imbalanced work between processes

How it Looks (Resource View)



CPU / GPU Utilization



Low – Moderate



Memory Bandwidth



Low – Moderate



Communication (MPI)



High (Near Max)



I/O Activity



Low – Moderate



Network / Communication is the limiting factor (bottleneck).

Signs You Are Communication-Bound

- ✓ High MPI time (% of total runtime)
- ✓ Many MPI calls with small message sizes
- ✓ Processes spend time waiting at barriers
- ✓ Scaling stops improving or even gets worse when adding more nodes
- ✓ Network bandwidth is close to saturation

Examples of Communication-Bound Simulations (Domains)

Large-Scale Climate Models (Atmosphere / Ocean)



Frequent global exchanges of large 3D fields.

Computational Fluid Dynamics (CFD)



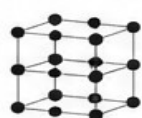
Decomposition requires halo/boundary data exchange.

Particle / N-Body Simulations



Particles interact across sub/domains; neighbor communication.

Electronic Structure (DFT, Plane-Wave)



Global operations (FFTs, reductions) are communication heavy.

Agent-Based / Multi-Physics Models



Frequent data exchange between components or regions.

How to Execute Communication-Bound Workloads Effectively



Reduce Communication Volume

Exchange only necessary data; compress or aggregate when possible.



Reduce Message Count

Use fewer, larger messages; avoid many small MPI sends.



Minimize Synchronization

Avoid global barriers; use asynchronous and non-blocking communication.



Improve Load Balance

Ensure similar work on all ranks to prevent waiting time.



Use Efficient Communication Patterns

Topology-aware mapping, collectives, overlap communication with computation.



Choose Right Scale & Decomposition

Not too many nodes; choose decomposition that reduces cross-node communication.



Key Takeaway: Minimize the need to talk, when you talk make it efficient, and never wait unnecessarily. Balance computation and communication for good scalability.

I/O-Bound Workloads

Scientific data movement to storage can dominate runtime.

I/O Scientific Output

Trajectories
Checkpoints
Mesh files
Logs
Visualization data
Restart files



I/O is the limiting factor when data cannot be written or read fast enough.

What is I/O-Bound?



The application spends most of its time waiting for data to be written to or read from storage.

Causes:

- Large data volumes (outputs, checkpoints)
- Many small files / metadata operations
- Slow storage bandwidth or IOPS
- Contention on shared file system

How it Looks (Resource View)



CPU / GPU Utilization



Low – Moderate



Memory Bandwidth



Low – Moderate



Communication (MPI)



Low – Moderate



I/O Activity



High (Near Max)



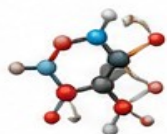
Storage I/O is the limiting factor (bottleneck).

Signs You Are I/O-Bound

- ✓ High time in I/O wait (iowait)
- ✓ Low CPU/GPU utilization
- ✓ High read/write times
- ✓ Many small file operations
- ✓ Performance improves with faster storage
- ✓ Scaling stops improving or gets worse when adding more compute nodes

Examples of I/O-Bound Simulations (Domains)

**Molecular Dynamics
(Long Trajectories)**



Frequent writing of trajectories and checkpoint files.

**Climate / Weather
Simulations**



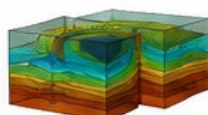
Large output files, frequent checkpoints, high-resolution data.

**Astrophysics
Simulations**



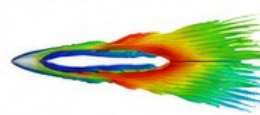
Large snapshots, particle data, volume data.

**Seismic / Geophysical
Simulations**



Large wavefield outputs, multiple time steps, checkpoints.

**High-Resolution CFD
(Unsteady)**



Large field data outputs, restart files, statistics.

How to Execute I/O-Bound Workloads Effectively



Reduce I/O Frequency

Write less often; increase output intervals and checkpoint intervals.



Use Efficient Data Formats

Use HDF5, NetCDF, ADIOS, XDMF, etc. instead of many small text files.



Parallel I/O

Use MPI-IO, parallel libraries and collective I/O.



Compress Data

Use compression (e.g., ZFP, gzip) to reduce data volume.



Use Fast Storage

Use SSD/NVMe tiers, burst buffers, or high-bandwidth file systems.



Use Local Scratch

Write to local SSD first, then stage out to shared storage.



Monitor and Tune

Use tools to profile I/O (Darshan, IOR, iostat, lfs). Identify hotspots and optimize.



Key Takeaway: Move only what is necessary, move it efficiently, and store it smartly. Optimize I/O to keep the simulation running at full speed.

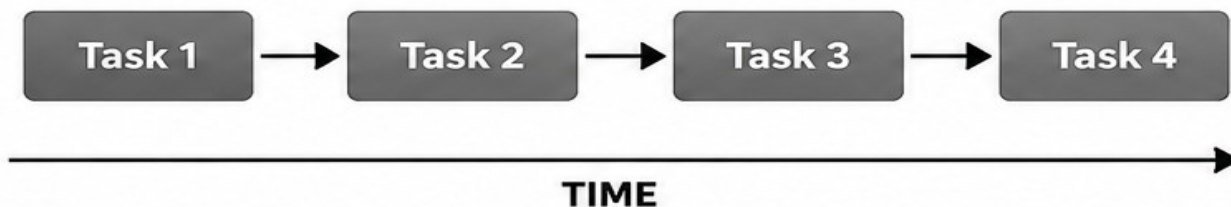
SERIAL vs PARALLEL EXECUTION

Parallel execution reduces time-to-solution, but introduces overhead.

SERIAL EXECUTION



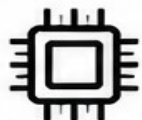
Tasks are executed one after another on a **single processor**.



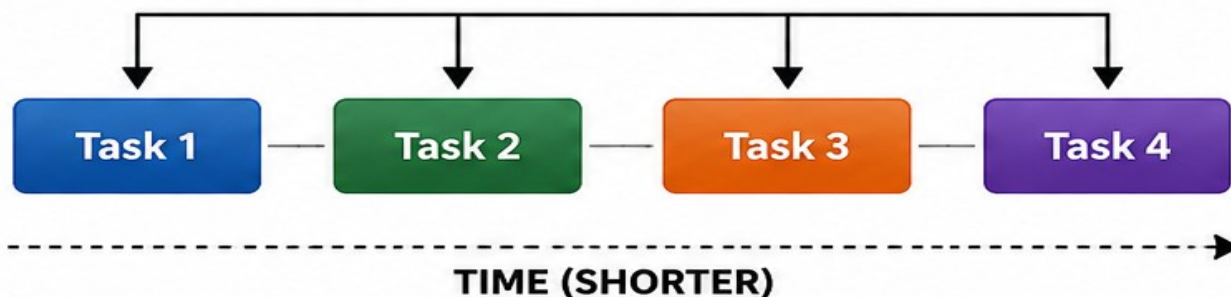
KEY POINTS

- Simple to implement
- Lower resource usage
- Longer total execution time
- Utilizes only one core

PARALLEL EXECUTION



Tasks are executed at the same time on **multiple processors**.



KEY POINTS

- Reduces time-to-solution
- Uses multiple cores/nodes
- Introduces overhead
- More complex to implement

PARALLEL OVERHEAD



COMMUNICATION

Data exchange between processors.



SYNCHRONIZATION

Waiting for other processors (barriers).



SCHEDULING

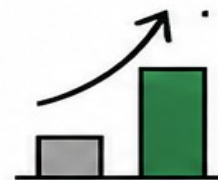
Distributing tasks to processors.



DATA MOVEMENT

Moving data between memory and nodes.

BENEFITS OF PARALLEL



Significant speedup for large problems when overhead is well managed.



TAKEAWAY:

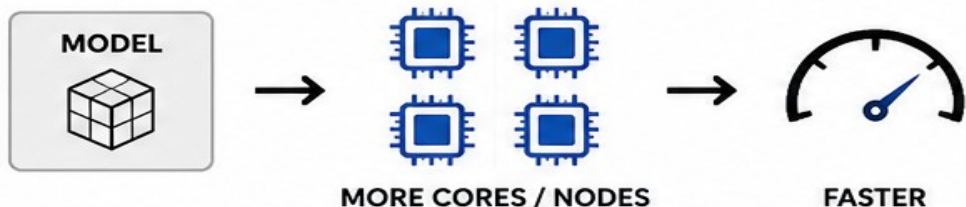
Use parallel execution to solve bigger problems faster, while managing overhead effectively.

HOW TO PERFORM PARALLELISM IN MODELS & SIMULATIONS

Goal: Goal: Solve the same problem faster or bigger by using multiple processors/nodes.

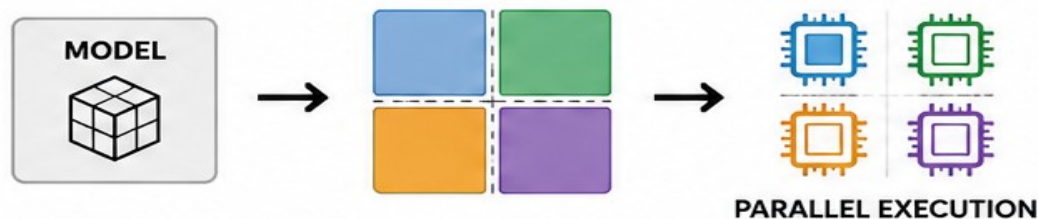
1 ADD MORE CORES / NODES

Run the same model on more resources.



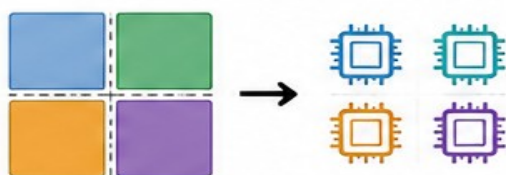
2 MODIFY / DECOMPOSE THE MODEL

Split the model so parts can run in parallel.



COMMON WAYS TO APPLY PARALLELISM

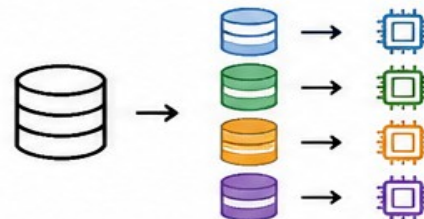
DOMAIN DECOMPOSITION



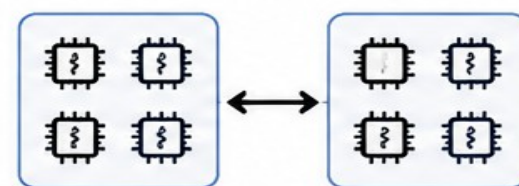
TASK DECOMPOSITION



DATA DECOMPOSITION



HYBRID (MPI + THREADS)



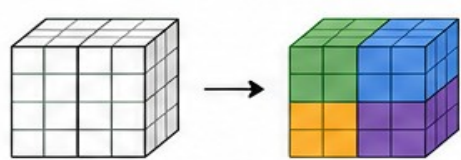
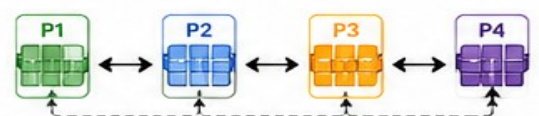
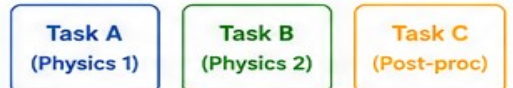
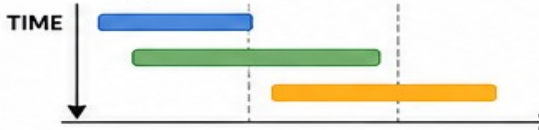
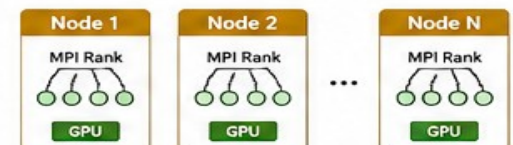
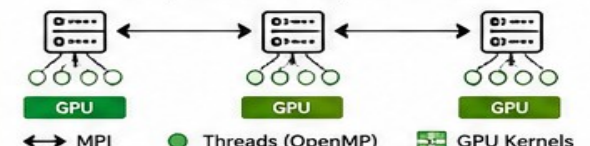
PARALLEL EXECUTION WORKFLOW



KEY TAKEAWAY

Add resources and/or restructure the model so work happens at the same time.

TYPES OF PARALLELISM IN MODELING & SIMULATION

TYPE	WHAT IT IS (VISUAL)	HOW IT WORKS	HOW TO DO IT (METHODS)	TOOLS / FRAMEWORKS
1 DOMAIN DECOMPOSITION  CFD, Weather, FEM, Molecular Dynamics	Split the physical domain 	Each processor works on a subdomain and exchanges boundary data. 	 Partition the domain  Distribute subdomains  Exchange boundaries  Iterate / Solve	 MPI  PETSc  OpenFOAM
2 TASK / FUNCTIONAL PARALLELISM  Multi-physics, Solver Stages	Different tasks run simultaneously 	Tasks execute in parallel and synchronize when needed. 	 Identify independent tasks  Run tasks concurrently  Synchronize / Exchange  Proceed to next stage	 MPI  OpenMP  Kokkos
3 ENSEMBLE / PARAMETER-SWEEP PARALLELISM  Different parameters, Monte Carlo, UQ	Many independent simulations run at the same time 	Each simulation is independent and runs on a separate core/node. 	 Vary parameters / inputs  Launch independent runs  Collect results  Analyze / Compare	 Slurm  Parsl  FireWorks
4 DATA PARALLELISM  Large data, ML, Data-driven sims	Large dataset split into chunks 	Each processor works on its data chunk in parallel. 	 Partition data  Distribute chunks  Process in parallel  Aggregate results	 Dask  Spark  NumPy
5 HYBRID PARALLELISM  Combine MPI + Threads + GPU	Multiple levels of parallelism 	MPI across nodes, threads within nodes, GPU for compute kernels. 	 Distribute with MPI  Use threads (OpenMP)  Accelerate with GPU  Coordinate efficiently	 MPI  OpenMP  CUDA



GOAL

Solve bigger problems faster and efficiently



BENEFIT

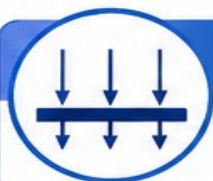
Higher speedup
Better resource usage



KEY

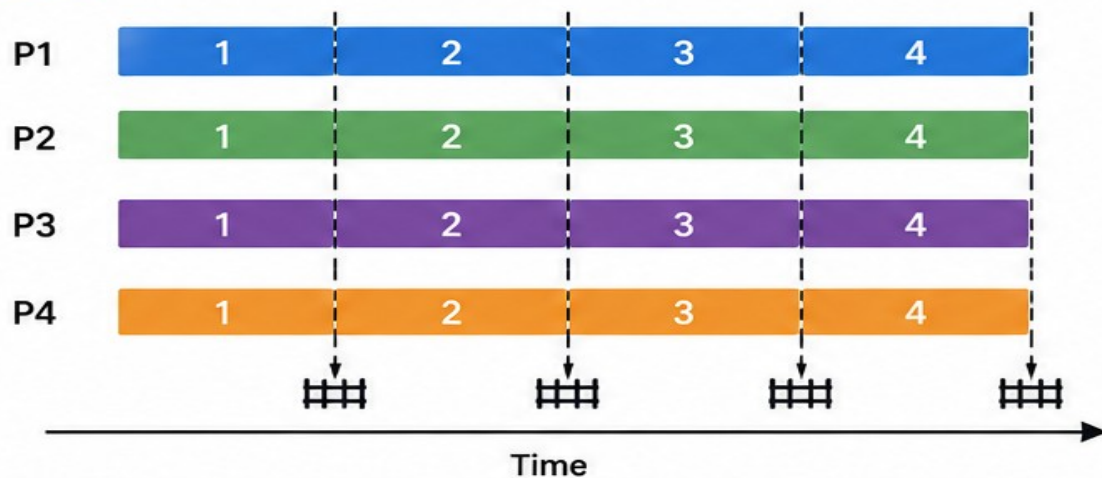
Choose the right type (or combination) for your simulation

SYNCHRONOUS vs ASYNCHRONOUS PARALLELISM



SYNCHRONOUS

Work in lockstep — wait at barriers



Simple



Predictable

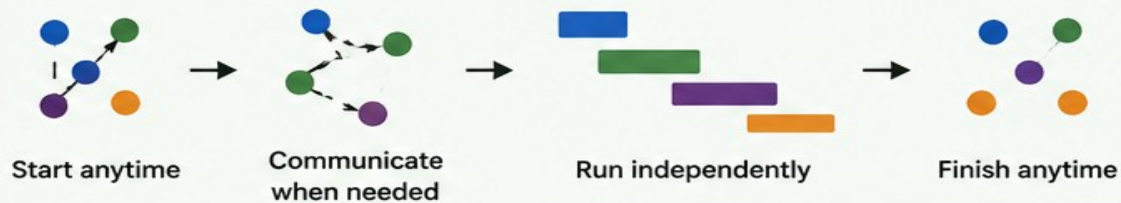
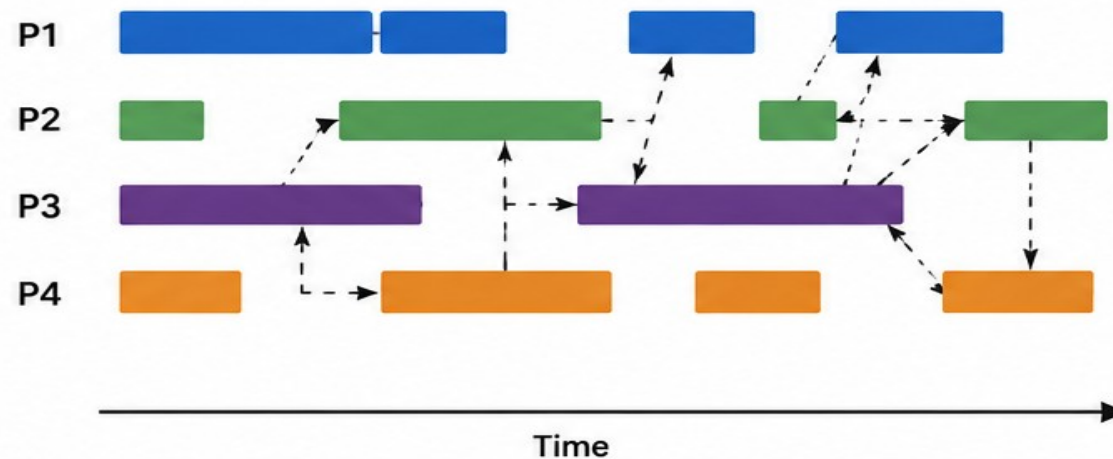


Idle time at barriers



ASYNCHRONOUS

Work independently — no waiting



Flexible



Better resource utilization



More complex



Use what fits your problem.

Often: synchronous within phases, asynchronous between phases.

1. WHAT IS SPEEDUP?

Speedup compares parallel runtime with the serial (baseline) runtime.

$$S = T_1 / \overline{T_P}$$

S = Speedup

T_1 = Serial (baseline) runtime

T_P = Parallel runtime with P processors

BASLINE (SERIAL)

1



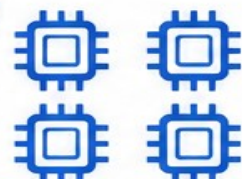
1 CPU

Runtime = 100 min



PARALLEL

2



4 CPUs

Runtime = 30 min

EXAMPLE

T_1 = 100 min

T_P = 30 min (4 CPUs)

$$\begin{aligned}\text{SPEEDUP} &= 100 / 30 \\ &= 3.33\times\end{aligned}$$



KEY TAKEAWAY: Four processors do not automatically produce 4× speedup.

2. PARALLEL EFFICIENCY

Efficiency shows how effectively the allocated processors are being used.

$$E = S / P$$

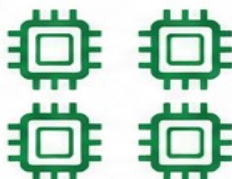
E = Efficiency

S = Speedup

P = Number of processors

EXAMPLE

1



4 Processors

Speedup (S) = 3.2×



EFFICIENCY

2



$$\begin{aligned}E &= S / P \\ &= 3.2 / 4 \\ &= 0.80 = 80\%\end{aligned}$$

WHAT DOES IT MEAN?

- ✓ 100% Perfect use of all processors
- ! < 100% Some resources are underused
- ✗ Low Poor utilization or high overhead



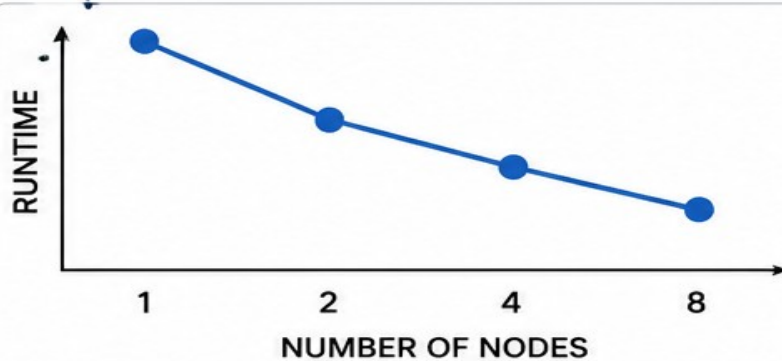
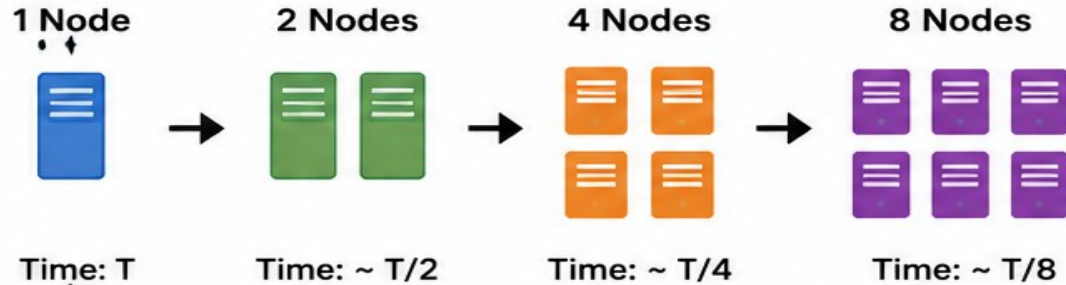
KEY TAKEAWAY: Higher efficiency means better use of allocated computing resources.

STRONG vs WEAK SCALING

STRONG SCALING



Same problem size
Increase resources to finish faster.



Runtime
decreases
as more
nodes are
added.

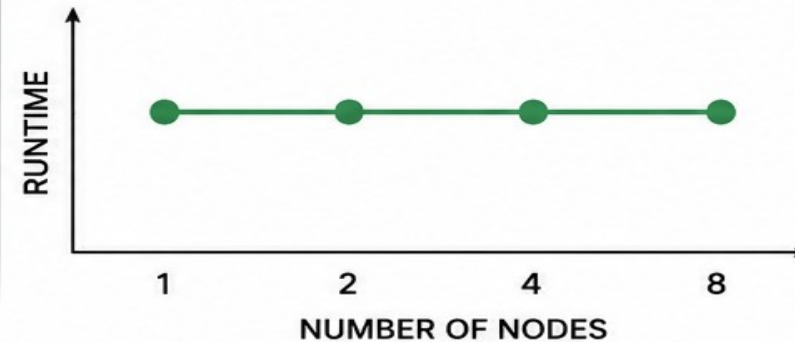
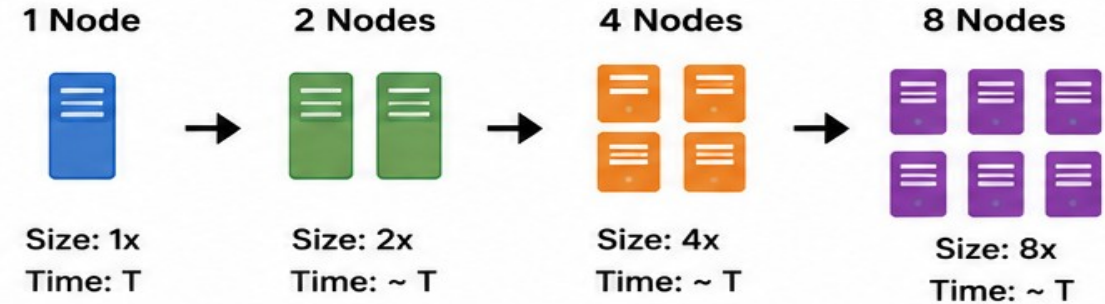


GOAL: Minimize runtime for
the same problem size.

WEAK SCALING



Problem size grows with resources
Keep runtime approximately constant.



Runtime
stays
approximately
constant.



GOAL: Solve proportionally larger problems
in approximately the same time.

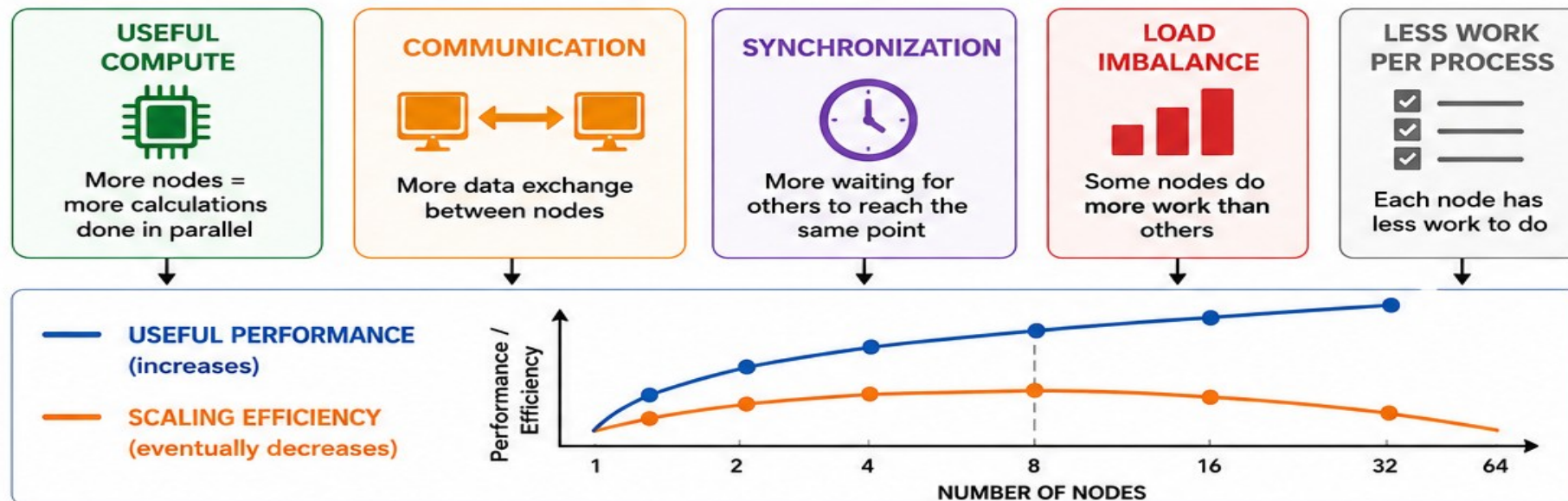


KEY IDEA: Strong scaling = **faster** for same size.

Weak scaling = **same time** for bigger size.

1. WHY SCALING EVENTUALLY STOPS

Every additional node adds both useful compute and overhead.



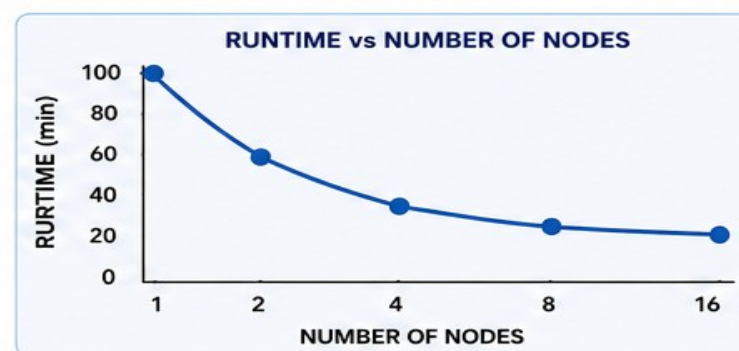
KEY IDEA:

Performance keeps improving, but efficiency eventually drops due to overhead.

2. MORE NODES ≠ BETTER PERFORMANCE

Resource efficiency matters as much as raw runtime.

NODES	RUNTIME (min)	SPEEDUP (vs 1 node)	EFFICIENCY	OBSERVATION
1	100	1.00×	100%	Baseline
2	55	1.82×	91%	Good scaling
4	30	3.33×	83%	Good scaling
8	22	4.55×	57%	Useful
16	21	4.76×	30%	Poor return on resources



KEY TAKEAWAY:

8 nodes often gives the better performance-per-resource trade-off.



CHOOSE WISELY

More nodes are not always better.



FIND THE SWEET SPOT

Choose the number of nodes that gives the best performance for the resources used.

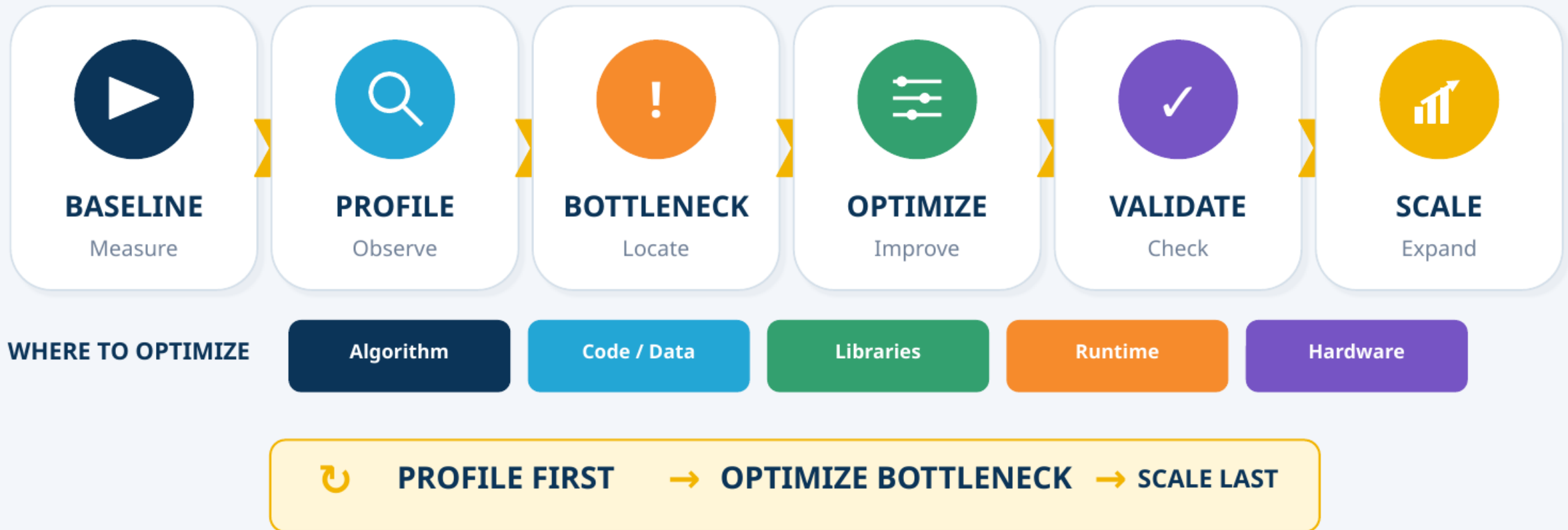


FOCUS ON EFFICIENCY

Aim for the best performance-per-resource, not just the lowest runtime.

System Optimization Method

Measure → Find → Improve → Verify → Scale



Choose Resources According to the Workload

Resource requests should reflect the dominant computational requirement.

CPU Intensive?

Cores / vectorization

GPU Accelerated?

GPU type / count

Memory Intensive?

RAM capacity / bandwidth

Communication Intensive?

Node count / interconnect

I/O Intensive?

Storage / scratch

Then choose: CPUs + GPUs + Memory + Nodes + Wall Time

CPU Optimization

Improve how algorithms map onto the processor architecture.



- Use optimized libraries where possible.
- Exploit multi-threading and SIMD/vectorization.
- Improve cache reuse and memory locality.
- Use appropriate compiler optimization flags.

GPU OPTIMIZATION

for Modeling & Simulation



Accelerate the dominant numerical work, not the whole simulation blindly.

1 Move the Right Computation to GPU

- Solver / matrix ops
- FFT / transforms
- Particle / cell ops
- Stencil / field updates



Which part dominates runtime?

2 Keep Simulation Data on GPU

- Minimize CPU ↔ GPU transfers
- Keep data resident
- Reuse intermediate results
- Transfer only outputs



Move computation to the data.

3 Increase Parallel Work

- Larger meshes / domains
- More particles
- Batch independent calculations
- Parallelize cells, particles, grid points



More work = better GPU utilization.

4 Use GPU-Optimized Libraries & Solvers



Linear Algebra
cuBLAS / cuSOLVER



Sparse Solvers
cuSPARSE



FFT / Transforms
cuFFT



Scientific Apps
(GROMACS, OpenFOAM, QE, etc.)

5 Optimize Numerical Configuration

- Solver & algorithm choice
- Mesh / domain decomposition
- Precision (FP64 / FP32 / mixed)
- Time step / iteration strategy
- Neighbor list / stencil settings



Maintain required scientific accuracy.

Simulation / Model



Math + Physics + Discretization

Find Hot Kernels



Profile



Identify bottlenecks
(solver, FFT, matrix, particles, etc.)

Map to GPU



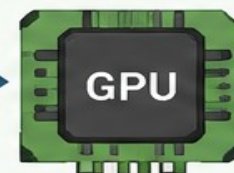
Large Work



Data on GPU



Libraries



GPU Execution



High Throughput



Fast Execution



Better Efficiency



Profile



Tune

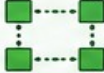
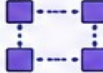
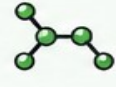


Validate
(accuracy)



Scale

GPU Optimization =
Parallel Numerical Work + Data Locality + Optimized Kernels

Optimization Area	What We Tune	GROMACS 	 Quantum ESPRESSO	 OpenFOAM
 Parallel Decomposition	Divide problem efficiently	 Domains / PME	 k-points / bands / FFT	 Mesh domains
 CPU ↔ GPU Mapping	Decide what runs on GPU	 Forces / PME / update	 GPU-enabled kernels	 GPU solver/backend
 MPI / Threads	Balance ranks and threads	 MPI + OpenMP	 MPI + OpenMP	 MPI processes
 Solver / Libraries	Use optimized numerical kernels	 PME / CUDA libraries	 FFT / BLAS / GPU libs BLAS	 Linear solver / preconditioner
 Data Movement	Keep useful data near GPU	 Coordinates / forces	 Wavefunctions / FFT data	 Matrix / field data
 Benchmarking	Find best configuration	 ns/day	 SCF / iteration time	 Solver / timestep time

GROMACS

- `gmxdmrun -s topol.tpr -deffnm md -ntmpi 4 \ -ntomp 8 -nb gpu -pme gpu -update gpu`
- `gmxdmrun -s topol.tpr -deffnm md -ntmpi 2 \ -ntomp 16 -nb cpu -pme gpu -update gpu`
- `gmxdmrun -s topol.tpr -deffnm md -ntmpi 1 \ -ntomp 32 -nb gpu -pme cpu -update cpu`

QUANTUM ESPRESSO

- `mpirun -np 64 pw.x -in scf.in -nk 4 -nband 8 \ -ntg 2 -ndiag 4`
- `mpirun -np 128 pw.x -in scf.in -nk 8 -nband 4 \ -ntg 4 -ndiag 8`
- `mpirun -np 32 pw.x -in scf.in -nk 2 -nband 16 \ -ntg 1 -ndiag 2`

OpenFOAM

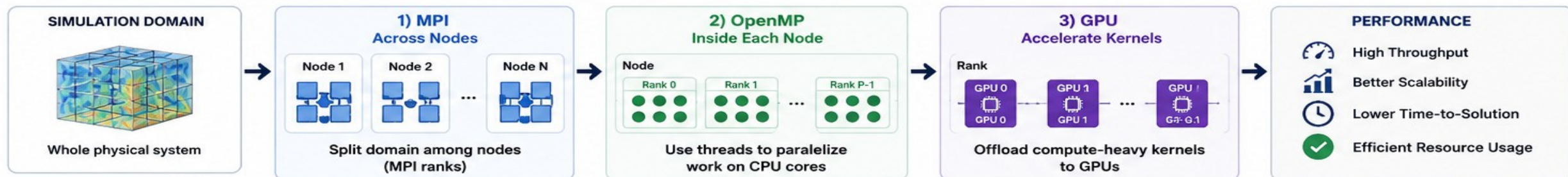
- `decomposePar -force`
`mpirun -np 64 icoFoam -parallel`
- `decomposePar -force`
`mpirun -np 128 pimpleFoam -parallel`
- `decomposePar -force`
`mpirun -np 32 simpleFoam -parallel`

-np : MPI ranks -ntmpi / -ntomp : MPI / OpenMP -nk : k-point pools -nband : band groups -ntg : FFT task groups -ndiag : diag groups

HYBRID / DISTRIBUTED HPC OPTIMIZATION

Optimize the same simulation across nodes, CPU cores and GPUs

Optimization Area	What We Tune	MPI — Across Nodes (Distributed Memory)	OpenMP — Inside Node (Shared Memory)	GPU — Accelerator (Compute-heavy)
 Problem Decomposition	Divide simulation work	 Domains / subdomains	 Loops / tasks	 Kernels / batches
 Parallel Configuration	Choose parallel workers	 MPI ranks	 Threads per rank	 GPUs / GPU streams
 Data Placement	Keep data close to compute	 Distribute data among nodes	 NUMA / shared memory	 Keep data resident on GPU
 Communication	Reduce data exchange	 Minimize MPI messages	 Reduce synchronization	 Minimize CPU ↔ GPU transfers
 Load Balance	Keep workers equally busy	 Balance work among ranks	 Balance thread workload	 Balance work across GPUs
 Memory Optimization	Improve locality / bandwidth	 Local node memory	 Cache / NUMA locality	 Coalesced GPU memory access
 Computation-Communication Overlap	Avoid waiting	 Non-blocking MPI	 Tasks while communication occurs	 Async transfer + kernels
 Resource Mapping	Match software to hardware	 Ranks per node	 Threads per rank	 GPU per rank / process
 Benchmarking	Find best combination	 1 → 2 → 4 nodes	 Threads scaling	 1 → 2 → 4 GPUs
 Scaling	Increase resources efficiently	 Strong / weak scaling	 Node-level scaling	 Multi-GPU scaling



EXAMPLE HYBRID SLURM COMMAND

```
srun -N 2 --ntasks-per-node=4 --cpus-per-task=8 --gpus-per-node=4 ./simulation_app
```

2 Nodes

4 MPI ranks
per node

8 OpenMP threads
per rank

4 GPUs
per node

Run tuned
simulation

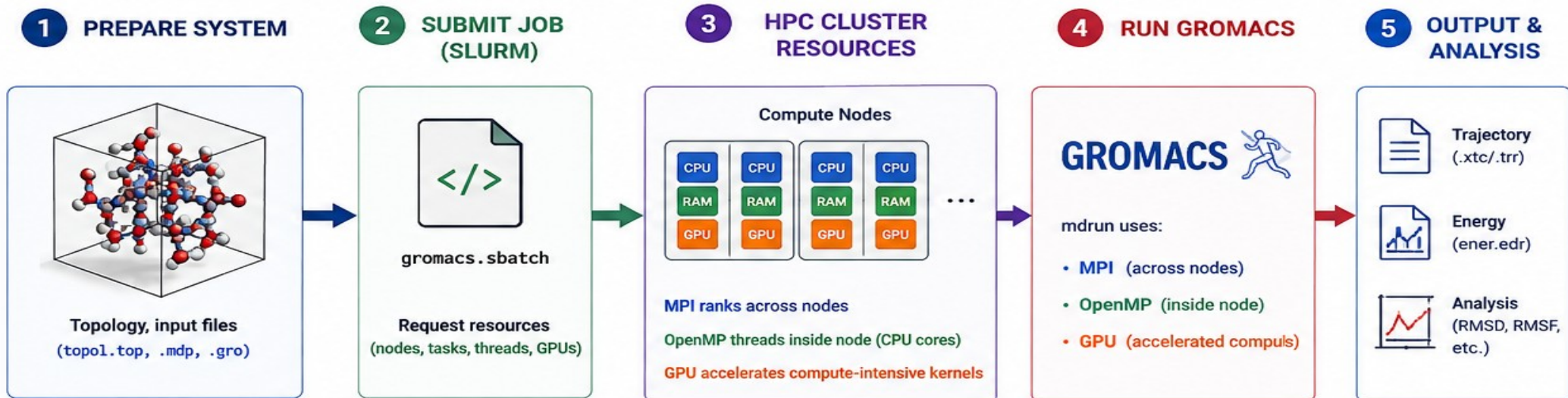
KEY MESSAGE



Hybrid Optimization =
Balanced Decomposition + Locality
+ Minimum Communication +
Correct CPU/GPU Mapping

Running & Optimizing GROMACS on an HPC Cluster

Right setup → Efficient use of resources → Faster simulations



EXAMPLE: GROMACS SLURM SCRIPT (gromacs.sbatch)

```
#!/bin/bash
#SBATCH -J gmx_test
#SBATCH -N 2
#SBATCH --ntasks-per-node=4
#SBATCH --cpus-per-task=8
#SBATCH --gres=gpu:4
#SBATCH --time=48:00:00
#SBATCH -p general
#SBATCH -o gromacs.out
#SBATCH -e gromacs.err
```

This example requests:

2 nodes

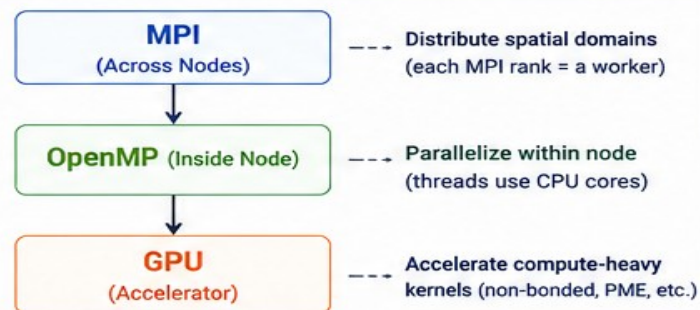
4 MPI ranks per node → 8 MPI ranks total

8 CPU cores (threads) per rank

4 GPUs per node (8 GPUs total)

Run: `srun gmx mdrun -s topol.tpr -deffnm md`

RESOURCE HIERARCHY IN GROMACS



WHAT WE TUNE

- MPI ranks**
(workers across nodes)
- OpenMP threads**
(cores per rank)
- GPU assignment**
(which GPU to use)
- Domain decomposition**
(load balance)
- PME settings**
(e.g., PME on GPU)
- Neighbor search**
(cutoff, update freq.)
- Output frequency**
(reduce I/O)



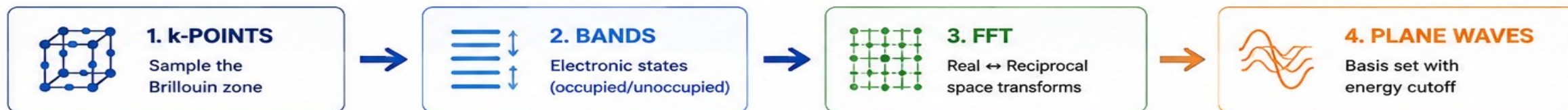
KEY PRINCIPLE

There is no universal best configuration.
Benchmark different combinations and choose the best for your system and hardware.



Running & Optimizing QUANTUM ESPRESSO on an HPC Cluster

Electronic-structure calculations expose multiple parallel dimensions.



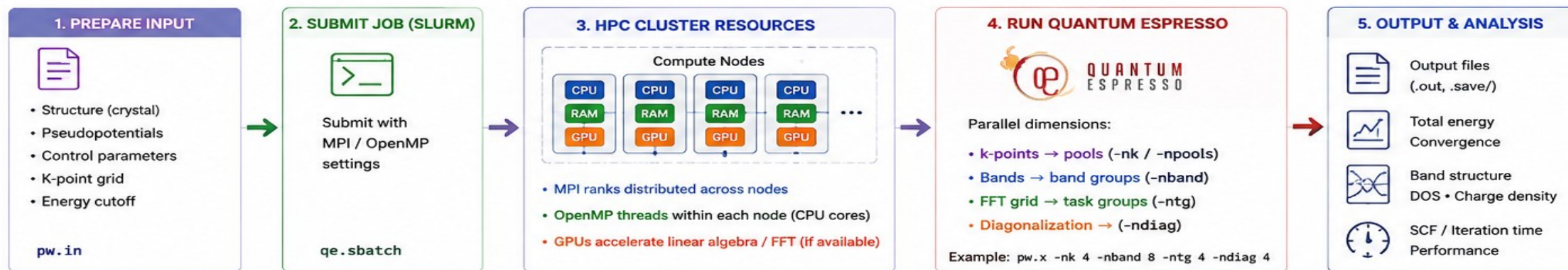
Performance depends on:
Atoms • Electrons • k-point grid • Cutoff



Memory & interconnect
performance can limit scaling.



Choose MPI/OpenMP decomposition
according to the problem size.



EXAMPLE: QE SLURM SCRIPT (`qe.sbatch`)

```
#!/bin/bash
#SBATCH -J qe_test
#SBATCH -N 2
#SBATCH --ntasks-per-node=4
#SBATCH --cpus-per-task=8
#SBATCH --gres=gpu:4
#SBATCH --time=24:00:00
#SBATCH -p general
#SBATCH -o qe.out
#SBATCH -e qe.err
```

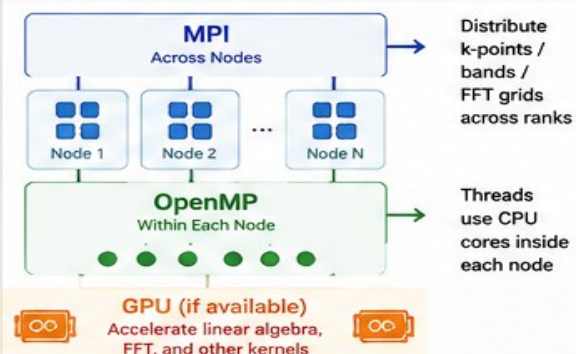
This example requests:

- 2 nodes
- 4 MPI ranks per node → 8 MPI ranks total
- 8 CPU cores (threads) per rank
- 4 GPUs per node (8 GPUs total)

Run:

```
srun pw.x -in pw.in -nk 4 -nband 8 -ntg 4 -ndiag 4
```

PARALLEL DECOMPOSITION IN QE



TUNING KEY PARAMETERS

	<code>-nk / -npools</code> Number of k-point pools
	<code>-nband</code> Band group size
	<code>-ntg</code> FFT task groups
	<code>-ndiag</code> Diagonalization groups
	<code>-ecutwfc / -ecutrho</code> Plane-wave cutoff (accuracy vs cost)



KEY PRINCIPLE

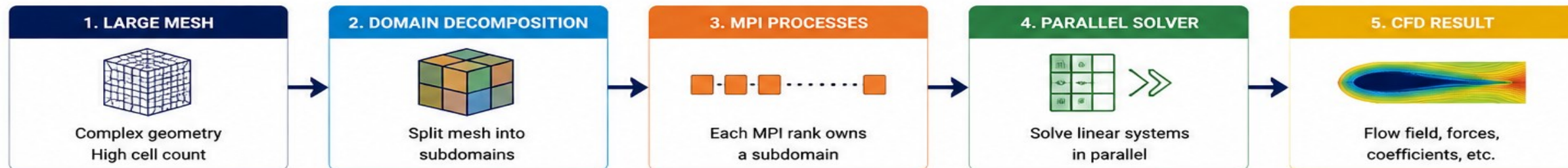
There is no universal best configuration.

Benchmark different decompositions and choose the one that gives the best performance for your system.



Running & Optimizing OpenFOAM on an HPC Cluster

Parallel CFD performance depends strongly on mesh decomposition and communication.



Balance cells across partitions.



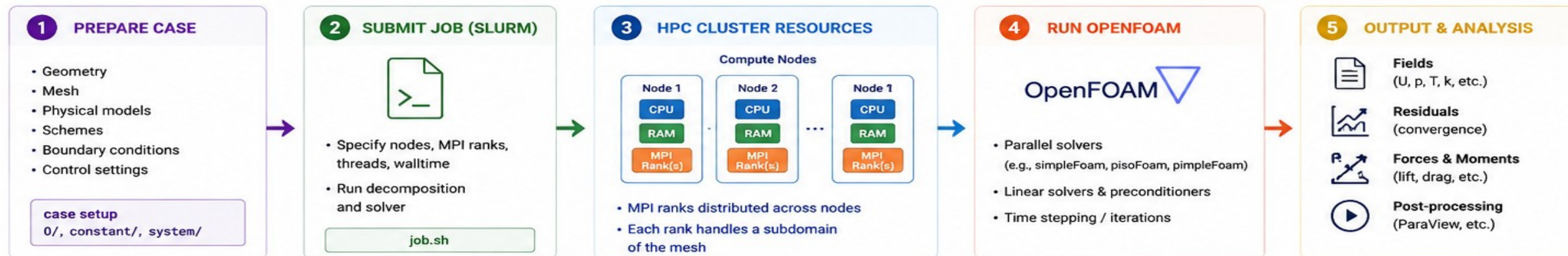
Minimize inter-domain communication.



Use appropriate solvers & preconditioners.



Poor decomposition causes processors to wait for each other.



EXAMPLE: OPENFOAM SLURM SCRIPT (job.sh)

```
#!/bin/bash
#SBATCH -J of_test
#SBATCH -N 2
#SBATCH --ntasks-per-node=4
#SBATCH --cpus-per-task=8
#SBATCH --time=24:00:00
#SBATCH -p general
#SBATCH -o log.%j.out
#SBATCH -e log.%j.err
```

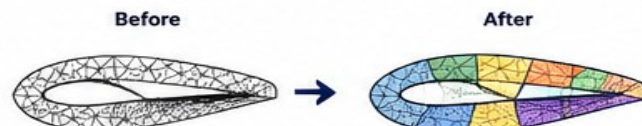
This example requests:

- 2 nodes
- 4 MPI ranks per node → 8 MPI ranks total
- 8 CPU cores (threads) per rank

Run:

```
srun decomposePar -force
srun simpleFoam -parallel
```

DOMAIN DECOMPOSITION (decomposePar)



- Use decomposeParDict to control method and number of subdomains
- Good decomposition → load balance + less communication

TUNING KEY PARAMETERS

	Mesh & Decomposition	decomposeParDict (method, number of subdomains)
	MPI Processes	Number of ranks (per node) Match problem size
	Solvers	fvSolution (solver type, tolerance)
	Preconditioners	fvSolution (e.g., GAMG, DIC, etc.)
	Communication	Minimize interfaces between subdomains



KEY PRINCIPLE

There is **no universal best configuration**.

Benchmark different decompositions, solvers, and process counts to get the best performance for your system and case.



SCHEDULER-AWARE SIMULATION

A GOOD HPC JOB REQUESTS WHAT IT NEEDS—NOT EVERYTHING AVAILABLE

! 1. AVOID OVER-REQUESTING

- Too many CPU cores
- Too much memory
- Too many GPUs
- Excessive wall time



→ Leads to long queues & wasted resources

✓ 2. REQUEST INTENTIONALLY

- Match resources to workload
- Use realistic wall time
- Choose correct partition
- Use appropriate job options



→ Better fit → Faster start → Efficient run

↑ 3. CLUSTER BENEFIT

- Shorter queues
- Higher utilization
- Fairer sharing
- Lower energy waste



→ Better for you and the whole community

HOW TO BE SCHEDULER-AWARE (GENERIC WORKFLOW)

1 Know Your Workload



Understand your code, CPU / Memory / GPU needs, I/O and scaling behavior

2 Check Cluster Info



Partitions & limits (info, show)
Policies & QoS
Max wall time

3 Estimate Resources



Choose:

- Nodes
- CPUs per task
- Memory
- GPUs (if needed)
- Wall time

4 Write a Good Job Script



Request only what you need
Use proper options

5 Submit & Monitor



Submit the job
Monitor progress
Watch for issues

6 Review & Optimize



Check performance
Adjust resources
Refine and repeat

WHAT YOU CAN DO

- ✓ Request the minimum resources that meet your needs
- ✓ Use the correct partition / QoS
- ✓ Set realistic wall time
- ✓ Use job arrays for parametric runs
- ✓ Use exclusive access only if needed
- ✓ Monitor usage and efficiency
- ✓ Scale tests: small → medium → large
- ✓ Cancel or requeue if stuck

EXAMPLE: GOOD JOB SCRIPT

```
#!/bin/bash
#SBATCH -J sim_run           # Job name
#SBATCH -p compute          # Partition
#SBATCH -N 2                # Number of nodes
#SBATCH --ntasks-per-node=4  # MPI ranks per node
#SBATCH --cpus-per-task=8    # CPU cores per rank (OpenMP)
#SBATCH --mem=0             # Use all memory on node
#SBATCH --gres=gpu:2         # 2 GPUs per node
#SBATCH -t 24:00:00         # Wall time (hh:mm:ss)
#SBATCH -o sim.%j.out        # Standard output
#SBATCH -e sim.%j.err        # Standard error

srun ./simulation_executable input.in
```

CHECK BEFORE SUBMIT

sinfo	→ View partitions & availability
scontrol show partition	→ View partition limits
sacctmgr show qos	→ View QoS limits
scontrol show node	→ View node resources
seff <jobid>	→ Check job efficiency
sacct -j <jobid>	→ Detailed job accounting
splot <jobid>	→ Visualize job performance

MATCH RESOURCES TO WORKLOAD

	CPU-bound	More CPU cores, threads, memory
	GPU-accelerated	Request GPUs only if code uses them
	Memory-bound	Request enough memory per node
	Long simulations	Set realistic wall time



KEY TAKEAWAY

Be considerate. Be efficient. Be scheduler-aware.

Right request → Fast start → Good performance → Happy cluster for everyone.

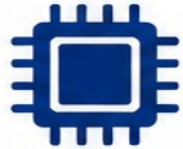


MONITOR YOUR HPC JOB



Observe utilization during execution before requesting additional resources.

1. CPU



Utilization / Load

- CPU % utilization
- Load average
- Idle time

2. GPU



Utilization / Memory

- GPU % utilization
- Memory usage
- Temperature
- Power usage

3. RAM



Capacity / Pressure

- Memory used / total
- Memory pressure
- Swap usage
- Cache / buffers

4. JOB



Runtime / State

- Job state (R, PD, CG, ...)
- Elapsed time
- Time remaining
- Node list

5. OUTPUT



Logs / Errors

- Output files
- Error messages
- Warnings
- Checkpoint files

USEFUL COMMANDS & TOOLS



CPU

top, htop, mpstat, sar -u

sacct, sstat



GPU

nvidia-smi

gpustat, DCGM



RAM

free -h, vmstat, sar -r

smem



JOB

squeue, sacct, sstat

seff <jobid>



OUTPUT

tail -f <log>, less <log>

grep -i error <log>

WHAT TO LOOK FOR



CPU

High CPU utilization?
Are cores busy or idle?



GPU

High GPU utilization?
Is GPU memory sufficient?



RAM

Memory nearly full?
Is there swapping?



JOB

Is the job running efficiently?
Any long wait or pending reasons?



OUTPUT

Any errors or warnings?
Are outputs being written?



**AFTER
COMPLETION**

Examine utilization
and performance.
Then decide whether to:



SCALE UP

Need more resources

OR



SCALE DOWN

Using more than needed

OR



KEEP SAME

Right balance

BUILD A BENCHMARKING STRATEGY



Benchmark progressively instead of jumping directly to a large allocation.

1. 1 NODE



- Establish baseline
- Check correctness
- Measure runtime
- Monitor resources

2. 2 NODES



- Increase resources
- Check scaling
- Measure runtime
- Monitor resources

3. 4 NODES



- Increase resources
- Check scaling
- Measure runtime
- Monitor resources

4. 8 NODES



- Increase resources
- Check scaling
- Measure runtime
- Monitor resources

5. COMPARE



- Compare results
- Analyze scaling
- Identify bottlenecks
- Assess efficiency

6. SELECT OPTIMAL

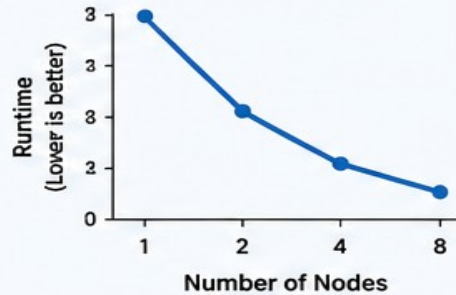


- Choose best config
- Balance speedup
- Maximize efficiency
- Optimize resources



RUNTIME

How much faster?

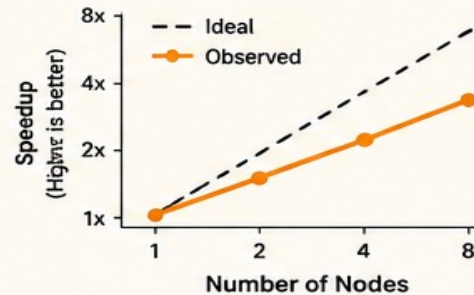


- Lower runtime
- Better performance
- Look for diminishing returns



SPEEDUP

How much parallel benefit?

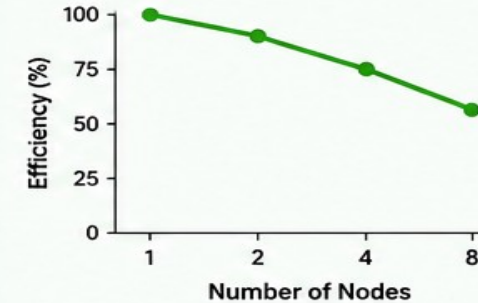


- Compare to ideal speedup
- Check scaling efficiency
- Identify bottlenecks



EFFICIENCY

How well are resources used?



- Higher efficiency is better
- Aim to keep efficiency high as you scale



KEY TAKEAWAY

Measure, analyze, and choose wisely.



MEASURE

Run benchmarks at different scales



ANALYZE

Compare runtime, speedup, efficiency



DECIDE

Select the optimal configuration



ACHIEVE

Best performance with efficient use



Start small. Measure. Understand scaling. Then scale smart.

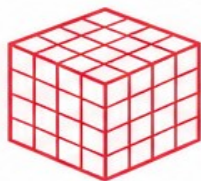
Right resources → Better performance → Efficient usage

FASTER IS NOT ALWAYS BETTER

Optimization must preserve scientific validity.

KEY FACTORS THAT AFFECT ACCURACY & COST

1 MESH RESOLUTION



Coarse meshes are cheaper but less accurate.

Refine enough to capture physics, not more.

2 SIMULATION TIME



Shorter runs reduce cost but may miss important phenomena.

Ensure sufficient physical time to reach steady state or capture key events.

3 NUMBER OF ATOMS



More atoms increase accuracy but also cost.

Use the minimum system size that represents the physics.

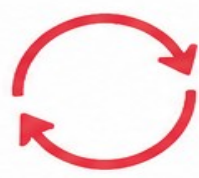
4 CUTOFF



Larger cutoff improves accuracy but increases computational cost.

Choose based on convergence testing.

5 ITERATIONS



More iterations improve convergence but increase runtime.

Run until results converge, not beyond.

6 NUMERICAL TOLERANCE



Tighter tolerances increase accuracy but cost more time.

Use the loosest tolerance that still gives reliable results.



**REDUCING THESE CAN MAKE THE RUN FASTER—
BUT MAY CHANGE THE SCIENTIFIC CONCLUSION.**



Always balance accuracy and cost.
Validate, verify, and report transparently.



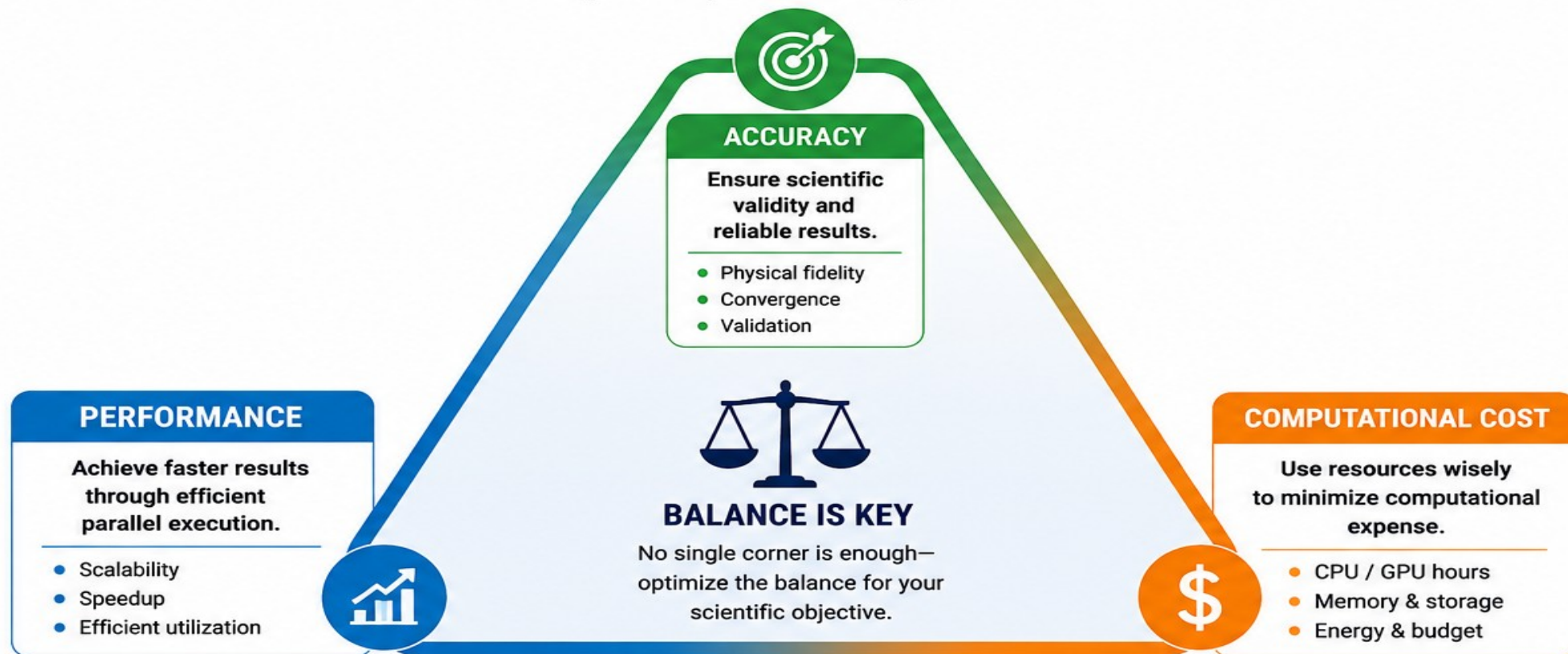
OPTIMIZATION MUST PRESERVE THE SCIENCE

Understand the physics → Test for convergence → Validate results → Report with confidence



THE HPC OPTIMIZATION TRIANGLE

Balance scientific accuracy, computational performance and resource cost.



THE GOAL



Understand the problem



Optimize the balance



Achieve reliable results



Deliver impact efficiently



**OPTIMIZE WISELY.
COMPUTE RESPONSIBLY.**

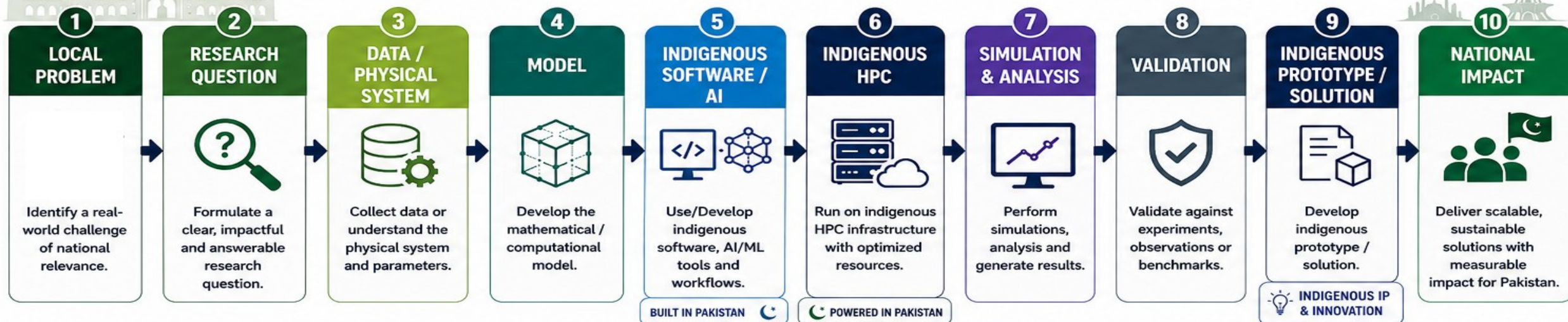
- ✓ Right balance → Better science
- ✓ Efficient use → Lower cost
- ✓ Smarter decisions → Greater impact



Better for you.
Better for everyone.

FROM LOCAL CHALLENGE TO INDIGENOUS HPC SOLUTIONS

Turn a local challenge into an **indigenous**, structured computational research pipeline for **Pakistan**.



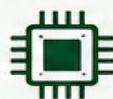
LOCAL RELEVANCE

Solve Pakistan's real challenges



SCIENTIFIC RIGOR

Strong methods, credible results



INDIGENOUS TECHNOLOGY

Home-grown tools, platforms & HPC



NATIONAL IMPACT

Self-reliant, scalable solutions for Pakistan



WHY INDIGENOUS?

- ✓ Data Security & Sovereignty
- ✓ Local Innovation & IP Creation
- ✓ Cost Effectiveness
- ✓ Solutions for Pakistan, by Pakistan



LET'S COLLABORATE TO SOLVE IT TOGETHER!



BRING YOUR DOMAIN EXPERTISE

Understand the problem deeper and better.



SHARE IDEAS & DATA

Build stronger, indigenous research questions.



CO-DEVELOP INDIGENOUS SOLUTIONS

Combine models, tools and methods together.



LEVERAGE INDIGENOUS HPC RESOURCES

Run larger, faster and smarter.



CREATE IMPACT FOR PAKISTAN

Deliver solutions that strengthen our society, industry and nation.

WE BUILD

- ✓ Indigenous Tools
- ✓ Indigenous Workflows
- ✓ Indigenous IP
- ✓ Indigenous Capability
- ✓ Indigenous Future



YOUR CHALLENGE



OUR RESOURCES



OUR EXPERTISE



INDIGENOUS HPC SOLUTIONS FOR PAKISTAN



LET'S CONNECT. LET'S COLLABORATE. LET'S BUILD INDIGENOUS HPC SOLUTIONS FOR A STRONGER PAKISTAN.

Possible Collaboration Models

There are multiple entry points for sustained research engagement.

1. Joint research projects

3. Faculty research

5. Scientific software development

7. Dataset development

9. Research proposals

2. MS / PhD thesis work

4. HPC benchmarking

6. Industrial problem solving

8. Joint publications

10. Training programs

Five Messages

A concise checklist for future scientific HPC work.

- 1 Model the right problem
- 2 Choose the right simulation method
- 3 Use HPC when computational scale demands it
- 4 Profile and optimize before scaling
- 5 **Turn computational capability into real-world impact**

SUMMER SCHOOL

AI Engineering for Supercomputing

Hands-on Training in AI Workloads, Distributed Training, Deployment, and Compiler Systems for HPC Platforms

Registration Fee

-  Professional: 40,000 Rs
-  Faculty: 30,000 Rs
-  Students: 5,000 Rs

Register Now!



 **August 29-30, 2026**

Topics

-  Introduction to Supercomputing Systems
-  AI-Based Vision Systems for HPC Workloads
-  Large Language Models and Multimodal AI
-  Distributed AI Training on HPC Clusters
-  AI Deployment and Inference Optimization for HPC Systems
-  AI for Local Industrial and Research Needs
-  AI Compiler Architecture, MLIR/LLVM, and Runtime Systems



Dr. Tassadaq Hussain
Director Centre for AI and Big Data



Dr. Mudassar Raza
Professor
Namal University,
Mianwali



Engr. Awais Baig
AI Systems Engineer
10x Engineers
Research Associate
Centre for AI and Big Data



Engr. Kamran Younis
Research Associate
Centre for AI and Big Data



Engr. Muhammad Usman
Research Engineer
Barcelona
Supercomputing
Centre



Namal University, Mianwali
(30Km Talagang Road)



Contact: +92 3022423109
ml.researcher@namal.edu.pk

Details: <https://tinyurl.com/namalai>

FROM LEARNING TO COLLABORATION

**YOUR PROBLEM + YOUR DOMAIN
EXPERTISE + CAID HPC
CAPABILITY**



COLLABORATIVE RESEARCH & INNOVATION

"The objective is not simply to use a supercomputer. The objective is to use computational science to solve problems that matter."

Thank You